TASK A: Application Familiarization

FailBlog is a simple Vaadin 8-based blog application backed by a JDBC in-memory database.

1.0 UI Layout

The app is a single-page layout with two panels:

Panels	**Contents**
Left	Article list, blog posts with a “>>” more button each
Right sidebar	Login form (or "Logged in" status) + Search form

2.0 Attack Surface

All input forms found are potential attack surfaces

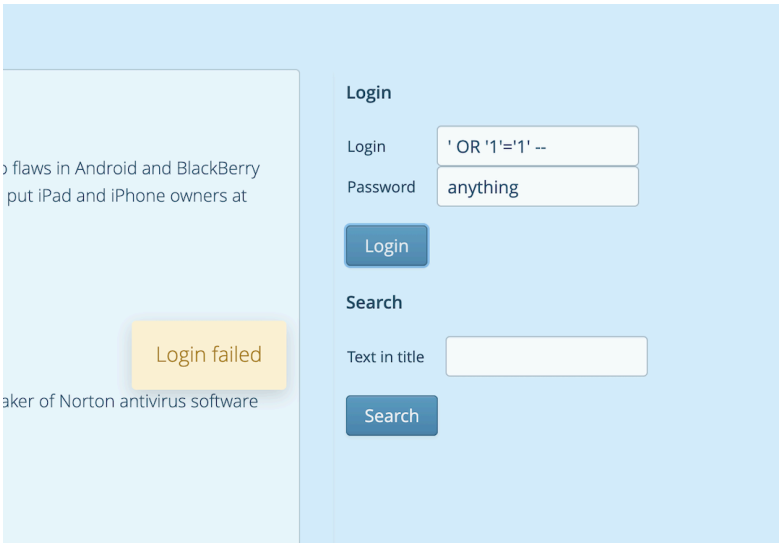
Form	**Fields**	**Location**
Login	Login (text), Password (text), Login button	Right sidebar (logged-out state)
Search	Text in title (text), Search button	Right sidebar (always visible)
Post Comment	Single text field, Post button	Inside each article view

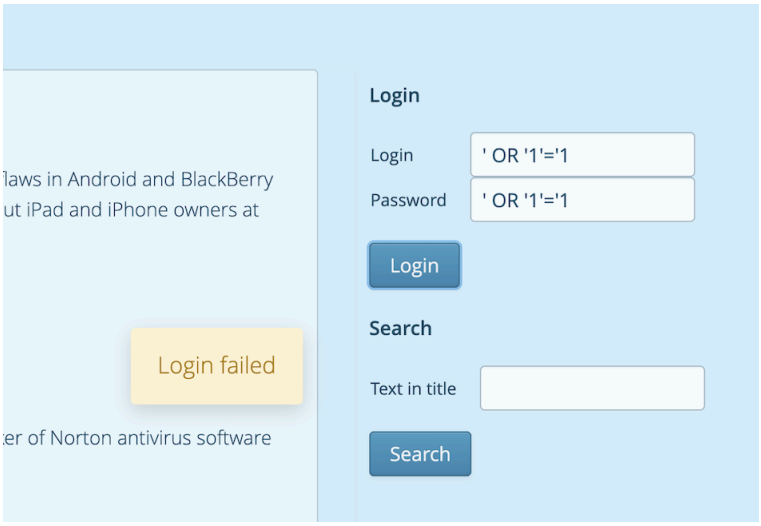
3.0 User Flow (with john / C2x6mZ)

1. On loading the application, the article list is shown, login form is visible on the right
2. Enter credentials, click **Login,** and the sidebar changes to **"Logged in as: John Doe"** + **Logout** button

3. Click ">>" more on any article, opens full article text and **Comments** section with existing comments, and a text input to post new ones
4. Search bar filters articles by title keyword

Task B: Login Without Credentials





- A typical login query looks like:

  SELECT * FROM users

  WHERE username = 'input_username'

  AND password = 'input_password';

- What happens with our input:

  Username: ' OR '1'='1'--

  Password: anything

- So the query becomes:

  SELECT * FROM users

  WHERE username = '' OR '1'='1' --

  AND password = 'anything';

- With the query above, the database should actually evaluate:

  SELECT * FROM users

  WHERE TRUE;

  In the case of a successful exploit, the following should be the result

  1. The query returns **all users**

  The application typically logs you in as:

  2. the **first user in the table**, often an **admin**

However, the exploit failed in our case study. This is likely because:

- Our payload does not align with the syntax the database system understands, e.g., Comment syntax mismatch

- The password is not part of the SQL query. The password check happens in the application logic

Adopting a Black Box Approach Strategy

In a black box scenario, we assume no access to the source code.

Step 1: Confirming the Vulnerability

By entering a single quote (') in the login field, we observe that the application returns a generic "Login failed". This suggests the input is being executed.

Step 2: Navigating the "Blind" Interface

The biggest hurdle is that the application is "Blind," i.e., it suppresses all SQL errors and shows a generic "Login failed" message for everything. An attacker cannot use standard "Error-based" counting (such as ORDER BY) because the UI never reveals whether a query succeeds or fails.

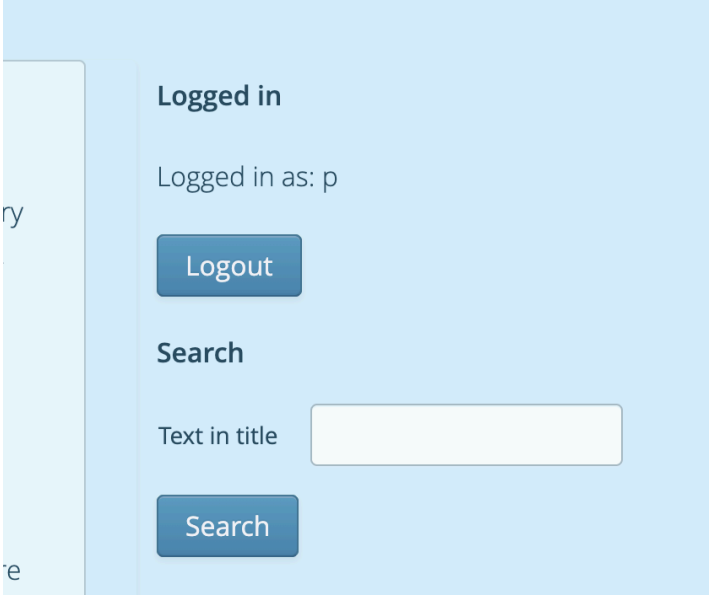
Step 3: Brute-force approach

Based on our knowledge that this is an in-memory DB, we don't just guess column counts; we also test for the required **FROM** syntax using the (VALUES(x)) row constructor.

To overcome this, instead of trying to find the column count first, I attempted to log in using a series of increasingly wide UNION payloads with a known password (e.g., 'p').

Phase	**Attempt (Login Field)**	**Attempt (Password Field)**	**Meaning**
A: In-Memory Probe	' UNION SELECT 'p' FROM (VALUES(0))--	p	Fails (Correct syntax, but wrong columns)
B: Iteration	' UNION SELECT 'p','p' FROM (VALUES(0))--	p	Fails (Wrong columns)

C: Iteration	' UNION SELECT 'p','p','p' FROM (VALUES(0))--	p	Fails (Wrong columns)
D: Success	' UNION SELECT 1,'p','p','p' FROM (VALUES(0))--	**p**	**LOGGED IN: Structure and Dialect both confirmed.**



I stopped iterating only when the "Logged in" state was reached, as depicted in the screenshot below. This single success signal confirms both the number of columns and the correct data types simultaneously.

Step 4: Determining Column Types and Order

We need to find which column holds the **password**. By "rotating" a known string through the columns:

- ' UNION SELECT 1, 'admin', 'p', 'Admin' FROM (VALUES(0))--
- Provided Password: p

When the application returns "Logged in", we have confirmed the schema: (Integer ID, String Login, String Password, String Name).

Why it is not "Straightforward"

This exploit is significantly more complex than the standard ' OR '1'='1 payloads taught in

introductory courses for three reasons:

1. The application code is most likely checking if the query returns more than one row: The common ' OR 1=1 attack returns every user in the database. Because the result set contains more than one row, this check triggers, returns null, and the login fails. You must return exactly one row to succeed.
In this specific assignment's environment (HSQLDB in-memory database), by using FROM (VALUES(0)), we create a virtual table with exactly one row.
2. In many simple examples, the password check is part of the SQL. Here, the database only fetches the user, and the Java application performs the comparison. You cannot "comment out" the password check; you must instead control the data that the database "hands over" to the Java code.
3. The developer most likely implemented a specific filter: login.replaceAll(" -- ", ""). In this case, our query must be carefuly crafted like this "…(VALUES(0))—" not like this "…(VALUES(0)) —"

Task C: Analysis of Unauthenticated Login Vulnerability

1.0 Vulnerability Identification

The vulnerability is a SQL Injection located wihin the findByLogin method of the UserDAO class.

```
public static User findByLogin(String login) {
    checkNotNull(login);

    // SQL injection protection
    login = login.replaceAll(" -- ", "");

    try {
        final Connection conn = getConnection();
        final Statement stmt = conn.createStatement();
        final ResultSet rs = stmt.executeQuery("SELECT * FROM user " +
                "WHERE login = '" + login + "'");
```

The application attempts to protect against SQL injection by removing the string " -- " (with spaces). However, this is insufficient because:

1. It does not handle single quotes ('), which are necessary to break out of the string literal.
2. It does not handle comments without spaces (e.g., -- or /*).
3. It uses string concatenation to build the query instead of parameterized queries.

Exploitation Mechanism
The login process in FailblogUI.java involves two steps:

1. Fetching a User object from the database using the provided login name.
2. Comparing the password from that User object with the password provided in the login form using String.equals().

An attacker can bypass this check using a UNION-based SQL injection.
Step-by-Step Exploit:

1. Target: Inject a synthetic row into the result set of the query.
2. Payload (Login Field): ' UNION SELECT 1,'admin','pass','Admin' FROM (VALUES(0))--
3. Payload (Password Field): pass

How it works inside the application:

The resulting SQL query executed by the server becomes:

```
SELECT * FROM user WHERE login = '' UNION SELECT 1,'admin','pass','Admin' FROM
(VALUES(0))--'
```

- Part 1 (WHERE login = ''): Returns no results — no user has an empty login, so 0 rows.
- Part 2 (UNION SELECT ... FROM (VALUES(0))): Injects exactly one fake record into the result set with a known password (pass).
- Part 3 (--): Comments out the trailing single quote left over from the original Java query template, preventing a syntax error.

- Java Logic: UserDAO.findByLogin retrieves this single fake record and returns it as a User object with password pass.

Validation: FailblogUI.login compares the form password (pass) with the User object's password (pass) — they match, login succeeds.

FROM user vs FROM (VALUES(0))

The findByLogin method includes a uniqueness guard:

```
// More results -> not unique -> can't choose
if (rs.next()) return null;
```

This means the query must return **exactly one row** for login to succeed.

Approach	**Rows returned by UNION SELECT**	**Outcome**
FROM user	One row per user in the table (e.g. 2 rows for 2 users)	Uniqueness check fails → null returned → login fails
FROM (VALUES(0))	Exactly **one** row (the inline single-row derived table)	Uniqueness check passes → fake user returned → login succeeds

(VALUES(0)) is a standard SQL construct that creates an anonymous inline table containing exactly one row with the value 0. It acts as a dummy source, guaranteeing only one result row regardless of what data exists in the real database.

The Problem with Meta-characters in the Context of this Vulnerability

Meta-characters are characters that have a special "functional" meaning to the language parser (in this case, the SQL interpreter) rather than being treated as literal data.

- **Single Quote (')**: The most critical meta-character here. It acts as a **delimiter**. It signals the end of the data string and the beginning of new SQL commands.
- **Comment Dash-Dash (--)**: Told the parser to ignore the rest of the line. This allowed the attacker to neutralize the developer's original closing quote.

In General

Meta-characters are problematic because they create Context Confusion.
When an application fails to properly separate the **Control Plane** (the commands/logic) from the **Data Plane** (the user's input), it becomes vulnerable. An attacker can use meta-characters to "leak" from the data plane into the control plane.
This is a fundamental issue across many types of vulnerabilities:

- SQL Injection: Meta-characters like ', --, and ; change the logic of the database query.
- Cross-Site Scripting (XSS): Meta-characters like <, >, and " change the structure of the HTML/JavaScript, allowing code execution in the browser.
- Command Injection: Meta-characters like ;, &, and | allow the execution of unexpected shell commands.

The most robust solution to this problem is to avoid string concatenation entirely and use Parameterized Queries (Prepared Statements). This ensures that user input is always treated strictly as data, regardless of what meta-characters it contains.

Task D: Determining the Number of Result Columns (Black-Box)

Technique Used: ORDER BY Column Index Injection
The ORDER BY keyword is used to sort the result-set in ascending or descending order.

If the index is within range, the database executes the query normally. If the index exceeds the number of columns, the database raises an error, which typically causes the application to return no results or behave differently.
This makes it possible to probe the column count from the outside, purely by observing application behaviour.

Attack Steps

The search form passes user input directly into the SQL query's WHERE clause via string concatenation (as identified in Task C). The raw search input becomes part of the query, so injecting:

```
%' ORDER BY N--
```

causes the server to execute:

```
SELECT * FROM article WHERE LCASE(title) LIKE '%' ORDER BY N--'%' ORDER BY id DESC
```

The '%' closes the LIKE string literal. ORDER BY N replaces the original ordering. -- comments out the rest of the original query, including the ORDER BY id DESC clause.

Payload entered in search box	**Resulting behaviour**
%' ORDER BY 1 --	Normal results returned
%' ORDER BY 2 --	Normal results returned
%' ORDER BY 3 --	Normal results returned
%' ORDER BY 4 --	Normal results returned
%' ORDER BY 5 --	Different behaviour — no results / error

The application returned results normally for column indices 1 through 4, but behaved differently (blank result / SQL error swallowed by the application) at index 5. The query that powers the article search returns exactly 4 columns.

Task E: Extracting All User Credentials via the Web Interface

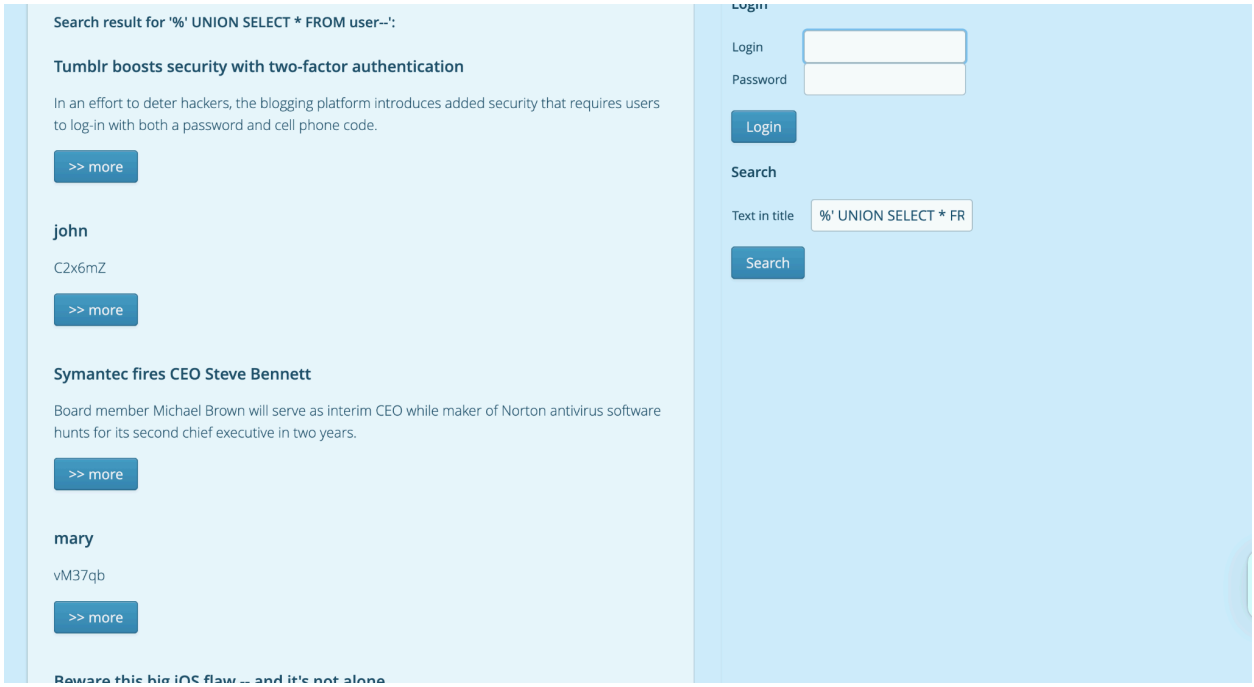
From Task D, we established that the article search query returns exactly 4 columns (determined by ORDER BY probing). A valid UNION SELECT must match this column count exactly.

We had no prior knowledge of the schema. The table name user was a common-sense guess: storing accounts in a table called user, users, account, or members is a well-known convention. We tried user first and it worked.

Using SELECT * instead of selecting specific columns meant we did not need to know individual column names or types upfront. SQL UNION requires compatible types in each position, but the only way to verify this black-box is to try the query:

- If results appear on screen → types were compatible.
- If no results appear (silent SQL error) → mismatched types

  The fact that UNION SELECT * FROM user-- immediately returned rows confirmed both the table name and type compatibility in a single attempt.



Task F: Identity Spoofing via SQL Injection

```
public static void addComment(final Comment comment, final long articleId) {
    // ...
    final Statement stmt = conn.createStatement();
    stmt.executeUpdate("INSERT INTO comment (text, article_id, user_id) "
            + "VALUES ('" + comment.getText() + "', " + articleId +
            ", " + comment.getUser().getId() + ")");
    // ...
}
```

The application directly concatenates the comment text (comment.getText()) into the INSERT statement. Since there is no escaping or sanitization, an attacker can "break out" of the text field and manipulate the subsequent fields (article_id and user_id).

To post a comment as a different user, an attacker follows these steps:

Step 1: Identify Target User ID

Using previous SQL injection techniques (like the one in the Search field), an attacker can find the id of the user they wish to impersonate.

- John Doe: ID 1
- Mary Glenn: ID 2

Step 2: Craft the Payload

The attacker crafts a comment string that closes the text quote, provides the desired article_id and user_id, and then comments out the rest of the original query.

Payload Template: [MESSAGE]', [ARTICLE_ID], [TARGET_USER_ID])--

Step 3: Execution

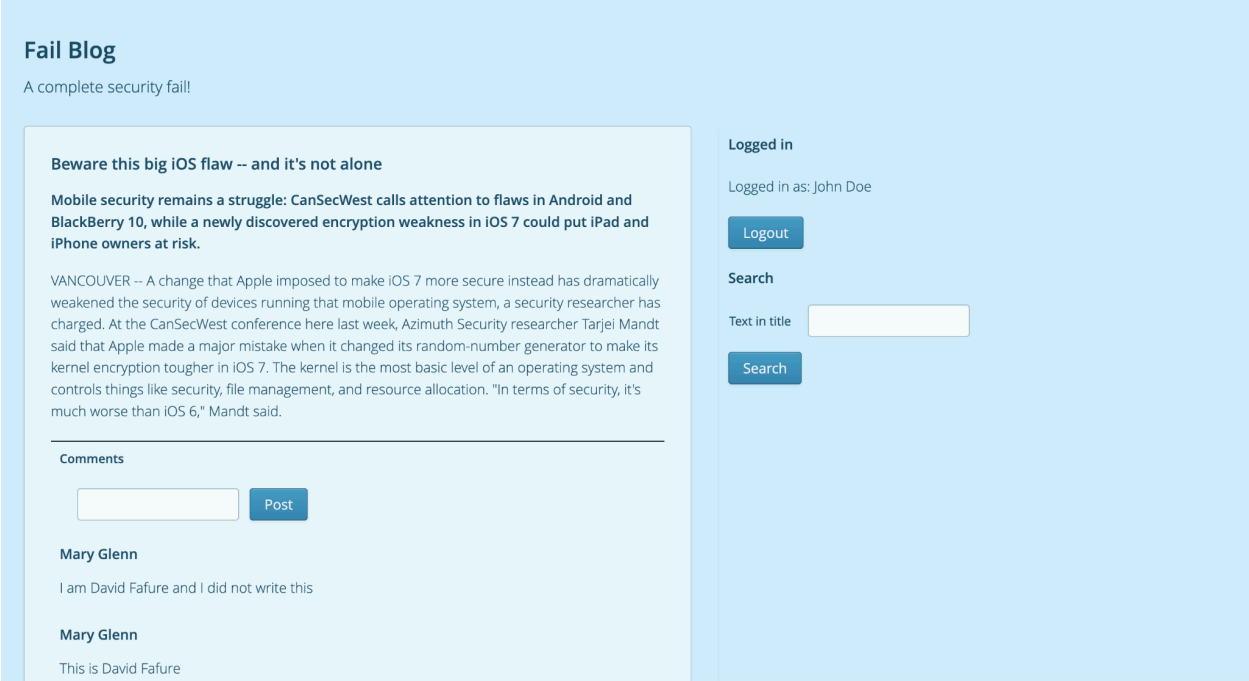
If the attacker is logged in (as any valid user) and wants to post a comment as John Doe (ID 1) on Article 3, they would enter the following in the comment box:

Comment Text: I am David Fafure and I didn't write this!', 3, 2)--

Resulting SQL Query:

```
INSERT INTO comment (text, article_id, user_id) VALUES ('I am David Fafure and
I didn't write this!', 3, 2)--', 3, [REAL_USER_ID])
```

The database executes the INSERT with user_id = 1. When the page reloads, the comment will be displayed as having been written by "John Doe".



Task G: Modifying the Database via Stacked Query Injection

Why the Comment Box Is the Right Attack Surface

The search form is vulnerable but only allows SELECT queries via executeQuery() — it cannot write data. The comment box, however, uses Statement.executeUpdate():

File: src/main/java/at/jku/ins/securecode/failblog/db/dao/CommentDAO.java

```
final Statement stmt = conn.createStatement();
stmt.executeUpdate("INSERT INTO comment (text, article_id, user_id) "
        + "VALUES ('" + comment.getText() + "', " + articleId +
        ", " + comment.getUser().getId() + ")");
```

Statement.executeUpdate() supports multiple semicolon-separated statements in a single call. This means we can terminate the intended INSERT early and append an entirely new SQL statement after a semicolon.

Technique Used: Stacked Query Injection

By injecting a semicolon (;) we can end the first statement and begin a second one that the database will also execute.

Attack Steps

The comment form is only accessible to authenticated users. So I logged in with John's credentials

Entered into the comment text box on any article page:

```
anything', 1, 1); INSERT INTO user (login, password, name) VALUES ('David',
'david', 'David Fafure');--
```

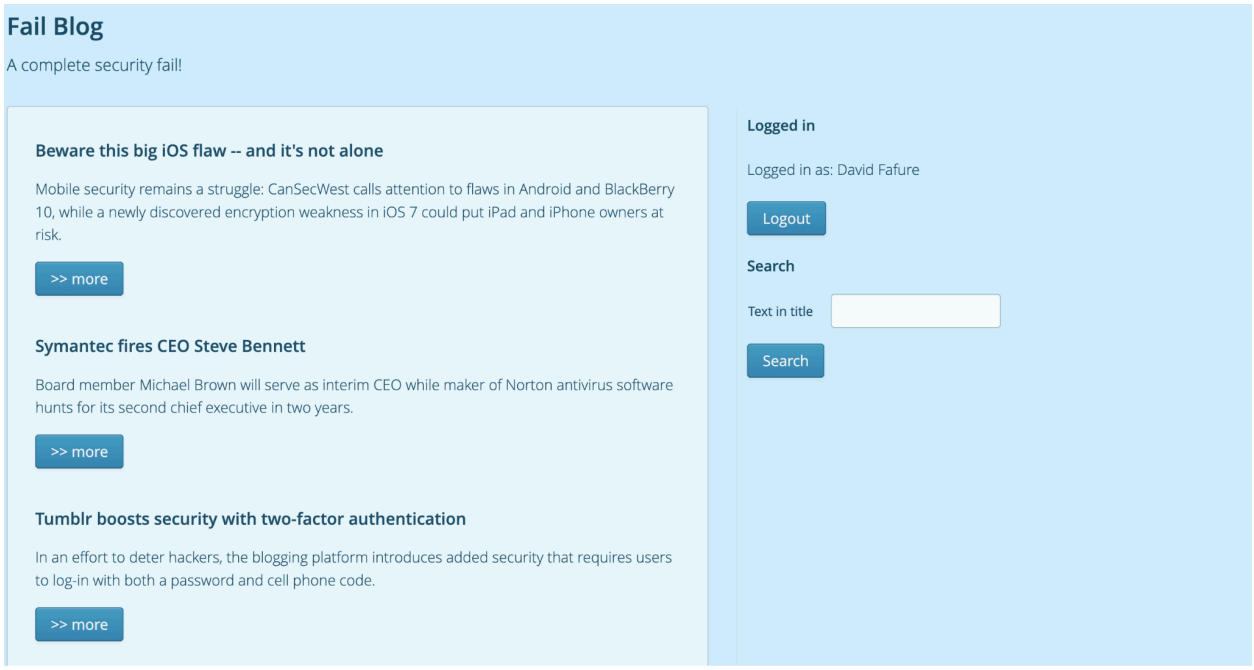
Resulting SQL on the server

```
INSERT INTO comment (text, article_id, user_id) VALUES ('anything', 3, 1);
INSERT INTO user (login, password, name) VALUES ('David', 'david', 'David
Fafure');--', [articleId], [userId])
```

After submitting, navigate to the login page and attempt to log in with:

- Login: David
- Password: david

Login succeeds, confirming the new account was created in the database.



Task H: Finding the Vulnerability That Allows Data Modification

The vulnerability that allows writing to the database is located in

CommentDAO.addComment:

File: *src/main/java/at/jku/ins/securecode/failblog/db/dao/CommentDAO.java*

```
final Statement stmt = conn.createStatement();
stmt.executeUpdate("INSERT INTO comment (text, article_id, user_id) "
        + "VALUES ('" + comment.getText() + "', " + articleId +
        ", " + comment.getUser().getId() + ")");
```

The comment text is concatenated directly into the SQL string with no escaping, sanitization, or parameterization. This gives an attacker two distinct injection capabilities

Capability 1: Rewriting field values (identity spoofing, Task F)

By closing the string literal early and supplying different column values, the attacker controls which user_id is stored

Capability 2: Stacked queries (arbitrary database changes, Task G)

By injecting a semicolon, the attacker terminates the first INSERT and appends an entirely new SQL statement:

Why It Works

The root cause is context confusion between the Data Plane and the Control Plane:

- The developer intends comment.getText() to be data.
- The database receives it as part of the SQL command string.
- The single quote (') meta-character terminates the data string, letting the attacker inject arbitrary SQL syntax.
- The semicolon (;) meta-character separates statements, letting the attacker execute entirely new commands.

Because Statement.executeUpdate() accepts multiple semicolon-separated statements and has no built-in protection against this, both injection styles succeed.

Why This Does Not Work via the Search Form

The search form calls ArticleDAO.findWhereTitleLike, which uses executeQuery():

File: *src/main/java/at/jku/ins/securecode/failblog/db/dao/ArticleDAO.java*

```
final PreparedStatement stmt = conn.prepareStatement("SELECT * FROM article "
        + "WHERE LCASE(title) LIKE "
        + "'%" + query.toLowerCase() + "%'"
        + "ORDER BY id DESC");
final ResultSet rs = stmt.executeQuery();
```

There are two reasons data modification is impossible here:

Reason	**Detail**
executeQuery() is read-only	JDBC's executeQuery() only processes SELECT statements. Any attempt to inject an INSERT, UPDATE, or DELETE (or a stacked query containing one) causes a SQLException

	— the database refuses to execute a data-modifying statement through a read-only call.
No write path in the result	Even if a UNION SELECT leaks data (as in Task E), the output is only rendered as article listings on screen. There is no mechanism in the search flow that writes anything back to the database.

In short: the search form can leak data but cannot modify it**.** The comment form is the only injection point that uses executeUpdate(), which both writes data and supports stacked queries.

Task I: Fixing All SQL Injection Vulnerabilities

The Universal Fix: Parameterized Queries

The root cause is the same across every vulnerable location: user input is concatenated into SQL strings. The only correct, comprehensive fix is to use PreparedStatement with? placeholders everywhere. This separates the SQL command structure from the data entirely, the database driver handles all quoting and escaping, so no meta-character in user input can ever alter the query's structure.

Blocklists (like replaceAll(" -- ", "") in UserDAO) are fundamentally broken: they try to predict what attackers will type instead of eliminating the injection channel. They must be removed.

Fix 1: CommentDAO.addComment (primary write vulnerability)

File: *src/main/java/at/jku/ins/securecode/failblog/db/dao/CommentDAO.java*

```
-final Statement stmt = conn.createStatement();
-stmt.executeUpdate("INSERT INTO comment (text, article_id, user_id) "
-        + "VALUES ('" + comment.getText() + "', " + articleId +
-        ", " + comment.getUser().getId() + ")");
+final PreparedStatement stmt = conn.prepareStatement(
+        "INSERT INTO comment (text, article_id, user_id) VALUES (?, ?, ?)");
+stmt.setString(1, comment.getText());
+stmt.setLong(2, articleId);
+stmt.setLong(3, comment.getUser().getId());
+stmt.executeUpdate();
```

Effect: The comment text is bound as a typed string value. Any single quotes, semicolons, or SQL keywords the user types are treated as literal characters, they cannot break out of the data plane. Stacked queries become impossible because the driver sends a single, pre-compiled statement to the database.

Fix 2: UserDAO.findByLogin (authentication bypass)

File: *src/main/java/at/jku/ins/securecode/failblog/db/dao/UserDAO.java*

```
-// SQL injection protection
-login = login.replaceAll(" -- ", ""); // REMOVE - blocklist is ineffective
-
-final Statement stmt = conn.createStatement();
-final ResultSet rs = stmt.executeQuery("SELECT * FROM user " +
-        "WHERE login = '" + login + "'");
+final PreparedStatement stmt = conn.prepareStatement(
+        "SELECT * FROM user WHERE login = ?");
+stmt.setString(1, login);
+final ResultSet rs = stmt.executeQuery();
```

Effect: Removes the flawed blocklist and replaces concatenation with a bound parameter. A UNION injection like ' UNION SELECT ... FROM (VALUES(0))-- becomes impossible, the entire string is treated as a literal login name, which matches no user and returns no results.

Fix 3: ArticleDAO.findWhereTitleLike (data extraction via search)

File: *src/main/java/at/jku/ins/securecode/failblog/db/dao/ArticleDAO.java*

```
-final PreparedStatement stmt = conn.prepareStatement("SELECT * FROM article "
-        + "WHERE LCASE(title) LIKE "
-        + "'%" + query.toLowerCase() + "%'"
-        + "ORDER BY id DESC");
+final PreparedStatement stmt = conn.prepareStatement(
+        "SELECT * FROM article WHERE LCASE(title) LIKE ? ORDER BY id DESC");
+stmt.setString(1, "%" + query.toLowerCase() + "%");
```

Effect: The ? placeholder is bound after the string is prepared, the % wildcards are part of the data value, not the SQL structure. Order-by probing (ORDER BY 5--) and UNION-based extraction (UNION SELECT * FROM user--) both become impossible because the injected syntax is treated as a literal search string.

Fix 4: ArticleDAO.findById

File: *src/main/java/at/jku/ins/securecode/failblog/db/dao/ArticleDAO.java*

```
-final Statement stmt = conn.createStatement();
-final ResultSet rs = stmt.executeQuery("SELECT * FROM article WHERE id = " +
id);
+final PreparedStatement stmt = conn.prepareStatement(
+        "SELECT * FROM article WHERE id = ?");
+stmt.setLong(1, id);
+final ResultSet rs = stmt.executeQuery();
```

Fix 5: UserDAO.findById

File: *src/main/java/at/jku/ins/securecode/failblog/db/dao/UserDAO.java*

```
-final Statement stmt = conn.createStatement();
-final ResultSet rs = stmt.executeQuery("SELECT * FROM user WHERE id = " +
id);
+final PreparedStatement stmt = conn.prepareStatement(
+        "SELECT * FROM user WHERE id = ?");
+stmt.setLong(1, id);
+final ResultSet rs = stmt.executeQuery();
```

Fix 6: CommentDAO.findByArticleId

File: *src/main/java/at/jku/ins/securecode/failblog/db/dao/CommentDAO.java*

```
-final Statement stmt = conn.createStatement();
-final ResultSet rs = stmt.executeQuery("SELECT * FROM comment "
-        + "WHERE article_id = " + articleId + " ORDER BY id");
+final PreparedStatement stmt = conn.prepareStatement(
+        "SELECT * FROM comment WHERE article_id = ? ORDER BY id");
+stmt.setLong(1, articleId);
+final ResultSet rs = stmt.executeQuery();
```

Fix 7: ArticleDAO.findAll

File: *src/main/java/at/jku/ins/securecode/failblog/db/dao/ArticleDAO.java*

```
-final Statement stmt = conn.createStatement();
-final ResultSet rs = stmt.executeQuery("SELECT * FROM article ORDER BY id
DESC");
+final PreparedStatement stmt = conn.prepareStatement(
+        "SELECT * FROM article ORDER BY id DESC");
+final ResultSet rs = stmt.executeQuery();
```

Note: This query contains no user input and is therefore not directly exploitable by SQL injection. However, replacing Statement with PreparedStatement here is still the correct practice, it is consistent with the rest of the codebase, removes the raw Statement entirely, and ensures the method is safe against future changes that might inadvertently introduce concatenation.